

Logo ENI Carthage
à insérer

Logo
SMARTOVATE
à insérer

École

Nationale d'Ingénieurs de Carthage
Département Génie Informatique

RAPPORT DE STAGE D'ÉTÉ

**Conception et développement d'une
application d'intelligence documentaire
basée sur une architecture RAG**

Réalisé

par

Mohamed Jerbi

2^e année du cycle ingénieur en Génie Informatique

Entreprise d'accueil
SMARTOVATE

Période
Du 28 juillet au 28 août 2026

Encadrant professionnel
À compléter

Encadrant académique
À compléter

Dédicace

À ma famille, pour sa présence et sa confiance. À mes enseignants, qui m'ont transmis la rigueur nécessaire au métier d'ingénieur. À toutes les personnes qui m'ont encouragé pendant ce stage et ont contribué, directement ou indirectement, à l'aboutissement de ce travail.

Remerciements

Je tiens à remercier l'équipe de SMARTOVATE pour son accueil et la confiance qui m'a été accordée pendant ce stage. Cette expérience m'a donné l'occasion de travailler sur un sujet actuel, à la rencontre du développement web, de l'intelligence artificielle et de la gestion documentaire.

Je remercie mon encadrant professionnel pour ses conseils, ses remarques et le temps consacré au suivi de mon travail. Son accompagnement m'a aidé à mieux organiser le projet et à prendre du recul devant les difficultés techniques. J'adresse également mes remerciements aux enseignants de l'École Nationale d'Ingénieurs de Carthage. Les connaissances acquises en Génie Informatique m'ont permis d'aborder ce projet avec une méthode structurée. Enfin, je remercie ma famille et mes proches pour leur soutien.

À personnaliser avant le dépôt : remplacer les mentions génériques par les noms et fonctions des encadrants.

Résumé

Les entreprises possèdent de nombreux documents internes, mais l'information utile reste souvent difficile à retrouver. L'objectif de ce stage est de concevoir et développer une application d'intelligence documentaire fondée sur l'approche RAG, ou génération augmentée par recherche. Avant de produire une réponse, l'application recherche les passages les plus proches de la question parmi les documents autorisés. La réponse est accompagnée de citations afin que l'utilisateur puisse vérifier son origine.

La solution comprend une interface React, une API Express écrite en TypeScript et un moteur de recherche hybride. Elle permet de téléverser plusieurs formats, d'extraire leur contenu, d'indexer les passages et de poser des questions en français. Quatre rôles structurent les droits : lecteur, éditeur, auditeur et administrateur. Le projet comprend aussi la double authentification, les signalements, l'audit, l'OCR des PDF scannés et la réinitialisation du mot de passe.

Le travail a été validé par des scénarios fonctionnels et de sécurité. Une version locale facilite la démonstration sans dépendre d'un service cloud. Une architecture cible est proposée pour une future mise en production.

Mots-clés : RAG, intelligence documentaire, recherche hybride, OCR, React, Express, contrôle d'accès.

Abstract

Companies store large amounts of internal documentation, yet useful information is often difficult to retrieve. This internship project focuses on the design and development of a document intelligence application based on Retrieval-Augmented Generation. Before generating an answer, the system retrieves relevant passages from documents the user is allowed to access. The answer includes citations so that its origin can be checked.

The solution combines a React interface, an Express API written in TypeScript, and a hybrid retrieval engine. It supports multiple formats, content extraction, indexing, French-language questions and source consultation. Access is organized through four roles : reader, editor, auditor and administrator. Authentication, two-factor authentication, knowledge-gap reports, audit logs, OCR for scanned PDF files and password reset are also included.

Functional and security scenarios were used to validate the work. A local mode makes demonstrations possible without cloud dependencies, while a target architecture is proposed for future production deployment.

Keywords : RAG, document intelligence, hybrid search, OCR, React, Express, access control.

Table des matières

Dédicace

Remerciements

Résumé

Abstract

Liste des abréviations v

Introduction générale 1

1 Cadre général du stage 2

1.1 Introduction 2

1.2 SMARTOVATE et le contexte professionnel 2

1.3 Contexte académique 2

1.4 Mission confiée 3

1.5 Organisation et méthode de travail 3

1.6 Planification synthétique 4

1.7 Conclusion 4

2 Analyse et spécification des besoins 5

2.1 Introduction 5

2.2 Problématique et étude de l'existant 5

2.3 Comparaison des approches 6

2.4 Acteurs et contexte 6

2.5 Besoins fonctionnels 6

2.6 Besoins non fonctionnels 7

2.7 Cas d'utilisation global 7

2.8 Description de cas d'utilisation essentiels 8

2.8.1 Ajouter et indexer un document 8

2.8.2 Poser une question et consulter les sources 8

2.9 Découpage du travail 9

2.10 Conclusion 9

3	Conception de la solution	10
3.1	Introduction	10
3.2	Architecture logique	10
3.3	Diagramme de composants	11
3.4	Modèle de domaine	11
3.5	Modèle de données	12
3.6	Séquence d'authentification	12
3.7	Ingestion et indexation	12
3.8	Recherche hybride et RRF	13
3.9	Séquence d'une question RAG	13
3.10	Contrôle d'accès par rôles	14
3.11	Cycle d'un signalement	14
3.12	Architecture locale et cible	14
4	Réalisation et validation	16
4.1	Introduction	16
4.2	Environnement technique	16
4.3	Authentification et gestion des profils	17
4.4	Gestion et traitement des documents	17
4.4.1	Extraction multi-format et OCR	17
4.5	Recherche, génération et citations	18
4.6	Signalements, audit et administration	18
4.7	Difficultés rencontrées et solutions	19
4.8	Stratégie de validation	19
4.8.1	Résultats fonctionnels	20
4.8.2	Résultats de sécurité	20
4.9	Mode d'exécution et déploiement envisagé	20
4.10	Limites et perspectives	21
4.11	Conclusion	21
	Conclusion générale	22
	Bibliographie et webographie	23
A	Scénario de démonstration	24
B	Matrice détaillée des droits	25
C	Informations à compléter	26

Table des figures

2.1	Diagramme de contexte	6
2.2	Cas d'utilisation global	7
3.1	Architecture logique en couches	10
3.2	Diagramme de composants	11
3.3	Diagramme de classes métier simplifié	11
3.4	Séquence d'authentification	12
3.5	Pipeline d'ingestion	12
3.6	Séquence d'une requête RAG	13
3.7	Capacités par rôle	14
3.8	Cycle d'une lacune documentaire	14
3.9	Architecture réellement utilisée en démonstration	14

Liste des tableaux

1.1	Découpage du stage	4
2.1	Comparaison des approches	6
2.2	Besoins fonctionnels	7
2.3	Cas d'utilisation : ajouter un document	8
2.4	Cas d'utilisation : interroger le corpus	8
2.5	Backlog synthétique du stage	9
3.1	Entités persistées	12
4.1	Technologies utilisées	16
4.2	Difficultés rencontrées pendant le stage	19
4.3	Tests fonctionnels	20
4.4	Tests de sécurité	20

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
BM25	Best Matching 25, classement lexical
CORS	Cross-Origin Resource Sharing
JWT	JSON Web Token
LLM	Large Language Model
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
SSE	Server-Sent Events
TOTP	Time-based One-Time Password
UML	Unified Modeling Language

Introduction générale

La documentation occupe une place importante dans une entreprise. Elle contient des procédures, des règles internes, des comptes rendus et des informations techniques. Pourtant, posséder ces fichiers ne garantit pas que l'information soit facile à retrouver. Un employé peut connaître le sujet recherché sans savoir dans quel document regarder ni quels mots exacts utiliser.

Les outils classiques répondent surtout à des mots-clés. Ils deviennent moins efficaces lorsque la question utilise un vocabulaire différent de celui du document. À l'inverse, un assistant général peut rédiger une réponse claire, mais il ne connaît pas les documents privés et peut fournir une information non vérifiée. L'approche RAG réunit les avantages des deux méthodes : retrouver les passages utiles, puis demander à un modèle de formuler une réponse à partir de ce contexte [1].

J'ai réalisé ce stage d'été chez SMARTOVATE du 28 juillet au 28 août 2026. Ma mission consistait à concevoir et développer une application web d'intelligence documentaire. L'utilisateur dépose des documents, les organise, pose une question en français et obtient une réponse accompagnée de citations. Le projet traite aussi la sécurité : les actions dépendent du rôle, les documents sont filtrés et les événements importants sont enregistrés.

La question directrice est la suivante : *comment construire, dans le temps limité d'un stage d'été, une application RAG démontrable, fiable et préparée pour une évolution vers la production ?* J'ai suivi une démarche progressive : étude du besoin, définition des acteurs, conception, réalisation par modules et validation.

Le premier chapitre présente le cadre du stage, la problématique et l'organisation du travail. Le deuxième formalise les besoins et les principaux cas d'utilisation. Le troisième explique les choix de conception. Enfin, le quatrième décrit la réalisation, les tests, les limites et les perspectives. Cette organisation reste volontairement proportionnée à un stage d'été de cinq semaines.

Cadre général du stage

1.1 Introduction

Ce premier chapitre situe le stage dans son environnement professionnel. Je présente d'abord l'organisme d'accueil et le besoin à l'origine du projet, puis je précise ma mission, le périmètre retenu et l'organisation des cinq semaines.

1.2 SMARTOVATE et le contexte professionnel

SMARTOVATE intervient dans la réalisation de solutions numériques et dans l'accompagnement de projets innovants. Mon stage s'inscrit dans cet environnement technique, avec un sujet qui réunit développement web, traitement documentaire, sécurité et intelligence artificielle. Mon sujet s'inscrit dans cette continuité, tout en se concentrant sur l'intelligence documentaire.

L'entreprise fournit un contexte réaliste : les informations sont réparties dans des fichiers variés et doivent rester accessibles uniquement aux bonnes personnes. Elle apporte également un cadre de validation, dans lequel une démonstration doit être comprise par un utilisateur non spécialiste. Enfin, le prototype doit préparer une évolution cloud sans présenter cette cible comme déjà déployée.

Le projet ne se limite donc pas à un chatbot. Il couvre la chaîne documentaire : dépôt, extraction, indexation, recherche, réponse, citation, contrôle d'accès et audit.

1.3 Contexte académique

Je suis étudiant en deuxième année du cycle ingénieur en Génie Informatique à l'ENI Carthage. Le stage mobilise plusieurs matières de la formation : développement web, programmation orientée objet, bases de données, génie logiciel, sécurité et modélisation. Il m'a aussi amené à étudier des notions nouvelles, notamment les embeddings, la recherche

vectorielle et la génération augmentée par recherche.

Ce travail m'a permis de mobiliser les connaissances acquises à l'ENI Carthage, mais aussi d'apprendre à cadrer un projet dont le périmètre était large par rapport à la durée du stage. J'ai donc privilégié un parcours complet et vérifiable plutôt qu'une accumulation de fonctions inachevées.

1.4 Mission confiée

La mission principale était de réaliser une application capable de répondre à des questions à partir d'un corpus interne. Elle comprend le téléversement de PDF, Word, Excel, Markdown, texte et CSV, l'extraction de leur contenu, la création de passages, la recherche, la génération d'une réponse française et l'affichage de citations.

Le périmètre inclut quatre profils. Le lecteur questionne le corpus. L'éditeur gère les documents. L'auditeur consulte les événements. L'administrateur dispose d'une vue globale et gère les comptes. L'application doit aussi fonctionner en mode local, afin que le parcours puisse être présenté même sans compte cloud ou crédit d'intelligence artificielle.

1.5 Organisation et méthode de travail

La durée de cinq semaines impose des priorités. J'ai commencé par analyser le dépôt et dresser l'état des fonctions. Cette étape a distingué les éléments terminés, partiels et non reliés au chemin principal. Le développement a ensuite été organisé en lots : interface, authentification, documents, moteur RAG, administration, sécurité et préparation au déploiement.

La validation a joué un rôle central. Une fonction n'était pas considérée terminée uniquement parce que son écran existait. Il fallait vérifier le comportement de l'API, les restrictions de rôle et les erreurs. Cette méthode a conduit à corriger l'inscription avec un rôle trop élevé, un téléversement insuffisamment protégé et l'absence d'isolation de certains documents.

1.6 Planification synthétique

Table 1.1 – Découpage du stage

Période	Travail	Résultat
Semaine 1	Analyse et besoins	Périmètre et risques identifiés
Semaine 2	Authentification et documents	Parcours stabilisés
Semaine 3	Recherche RAG et citations	Réponse sourcée
Semaine 4	Administration et sécurité	Rôles, audit, tickets
Semaine 5	OCR, email, tests et documentation	Version démontrable

Cette planification reste une synthèse. Les corrections issues des tests ont parfois obligé à revenir sur un lot précédent. Cette itération est normale : une autorisation manquante découverte en validation modifie à la fois l'API, l'interface et la documentation.

1.7 Conclusion

Le stage associe un besoin métier simple à expliquer et une réalisation technique variée. Cette combinaison justifie une conception précise avant la présentation du code.

Analyse et spécification des besoins

2.1 Introduction

Avant de développer, j'ai traduit le sujet en besoins observables. Cette étape m'a aidé à éviter deux solutions incomplètes : un simple moteur de recherche qui ne formule aucune réponse, ou un assistant qui répond sans montrer ses sources.

2.2 Problématique et étude de l'existant

Dans une organisation, l'information est dispersée entre procédures PDF, fichiers Word, feuilles Excel et notes. La recherche par nom de fichier oblige l'utilisateur à connaître à l'avance le vocabulaire du document. Une recherche plein texte repère bien une expression exacte, mais comprend mal une reformulation. Un modèle général est plus souple, mais ne connaît pas les informations privées et peut inventer.

La solution doit répondre à trois exigences : retrouver le sens d'une question, produire une réponse lisible et fournir une preuve documentaire. L'enjeu n'est pas de remplacer les documents, mais de créer un accès plus direct à leur contenu.

2.3 Comparaison des approches

Table 2.1 – Comparaison des approches

Critère	Recherche classique	LLM seul	RAG
Langage naturel	Limité	Très bon	Très bon
Documents privés	Oui	Non par défaut	Oui
Réponse rédigée	Non	Oui	Oui
Sources	Fichiers trouvés	Rarement	Citations
Risque d'invention	Faible, sans synthèse	Élevé	Réduit

L'approche RAG a été retenue parce qu'elle garde le document au centre. Le risque d'erreur n'est pas supprimé, mais la citation permet de vérifier. Une réponse hors périmètre doit être refusée clairement au lieu d'être complétée par une supposition.

2.4 Acteurs et contexte

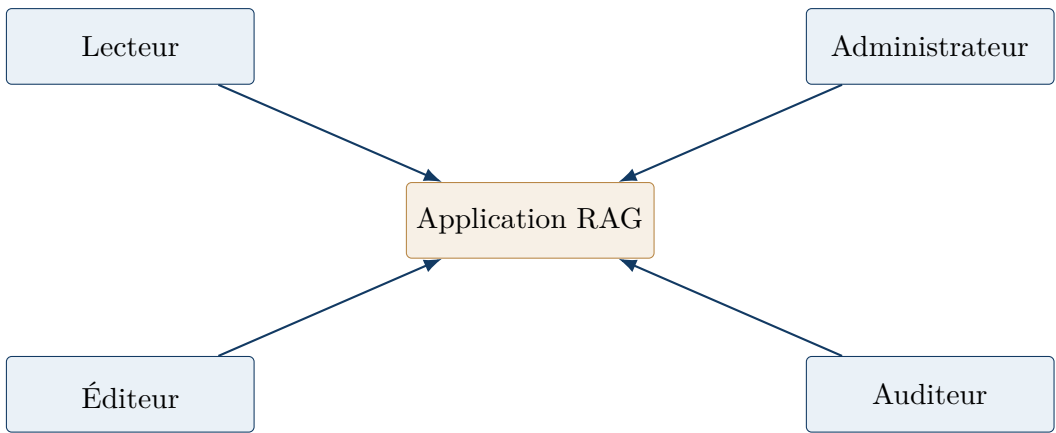


Figure 2.1 – Diagramme de contexte

Le lecteur consulte et signale une lacune. L'éditeur ajoute et organise. L'auditeur lit les journaux sans modifier le corpus. L'administrateur gère les comptes et les statistiques. Cette séparation suit le moindre privilège [2].

2.5 Besoins fonctionnels

Table 2.2 – Besoins fonctionnels

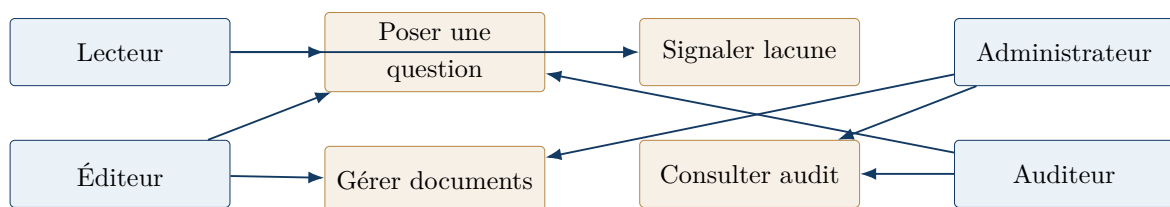
ID	Fonction	Description
BF01	Authentification	Inscription, connexion, déconnexion, refresh et reset.
BF02	Double authentification	Code TOTP facultatif.
BF03	Documents	Upload, dossiers, renommage, suppression et réindexation.
BF04	Extraction	PDF, DOCX, XLSX, MD, TXT, CSV et OCR.
BF05	Chat RAG	Recherche, flux SSE et citations.
BF06	Hors périmètre	Refus si aucune source pertinente.
BF07	Signalement	Création et suivi d'un ticket.
BF08	Administration	Statistiques, rôles et suspension.
BF09	Audit	Recherche et export CSV.
BF10	Profil	Préférences, 2FA et suppression.

2.6 Besoins non fonctionnels

La sécurité exige un jeton valide pour les opérations privées, une autorisation contrôlée côté serveur et une validation des fichiers. La traçabilité demande l'enregistrement des requêtes et événements sensibles. L'interface doit rester lisible sur ordinateur et mobile, dans les thèmes clair et sombre.

Pour la performance, le flux SSE permet d'afficher les premiers mots sans attendre la réponse complète. La maintenabilité repose sur TypeScript, la séparation frontend/backend et des entrées validées. Enfin, la portabilité impose un mode local sans service distant et des connecteurs cloud configurables.

2.7 Cas d'utilisation global

**Figure 2.2** – Cas d'utilisation global

Les critères sont observables : un lecteur non connecté est redirigé ; un lecteur connecté reçoit une erreur devant une opération d'administration ; un éditeur dépose un fichier et

voit son statut devenir indexé ; une question couverte produit des sources ; une question hors sujet produit un message prudent.

2.8 Description de cas d'utilisation essentiels

2.8.1 Ajouter et indexer un document

L'acteur principal est l'éditeur. Après authentification, il choisit un dossier et dépose un fichier autorisé. Le système vérifie sa taille et son extension, enregistre ses métadonnées, extrait le texte puis crée les passages utilisés par la recherche. Le cas se termine lorsque le document porte l'état « indexé ». Si l'extraction échoue, le document reste visible avec un état d'erreur afin que l'utilisateur puisse comprendre le problème et relancer le traitement.

Table 2.3 – Cas d'utilisation : ajouter un document

Élément	Description
Acteur	Éditeur ou administrateur
Précondition	Session valide et droit de téléversement
Scénario principal	Choisir un dossier, déposer le fichier, valider, extraire, découper et indexer
Alternative	Rejet si le format ou la taille n'est pas accepté
Postcondition	Document indexé et disponible dans le corpus autorisé

2.8.2 Poser une question et consulter les sources

Tout utilisateur connecté peut poser une question. L'API détermine d'abord les documents qu'il a le droit de consulter. Le moteur classe ensuite les passages, puis génère une réponse uniquement si le contexte est suffisant. L'utilisateur peut ouvrir chaque citation pour vérifier le document et l'extrait utilisés. Si aucun passage n'est pertinent, l'application signale simplement que l'information n'a pas été trouvée.

Table 2.4 – Cas d'utilisation : interroger le corpus

Élément	Description
Acteur	Lecteur, éditeur, auditeur ou administrateur
Précondition	Session valide et au moins un document accessible
Scénario principal	Saisir la question, rechercher, classer, générer puis afficher les citations
Alternative	Aucune source pertinente : réponse prudente sans génération libre
Postcondition	Question, latence et sources enregistrées pour le suivi

2.9 Découpage du travail

Pour une durée d'un mois, un backlog de plusieurs dizaines de récits aurait donné une précision artificielle. J'ai regroupé le travail en cinq lots correspondant aux semaines du stage.

Table 2.5 – Backlog synthétique du stage

Lot	Objectif vérifiable	Priorité
L1	Analyser l'existant, lancer le projet et recenser les risques	Haute
L2	Stabiliser l'authentification et la gestion documentaire	Haute
L3	Relier l'indexation, la recherche hybride et les citations	Haute
L4	Vérifier les rôles, l'audit et les signalements	Haute
L5	Ajouter l'OCR, le reset, les tests et la documentation	Moyenne

2.10 Conclusion

La qualité d'une application RAG dépend autant de l'ingestion, des droits et des sources que du modèle génératif.

Conception de la solution

3.1 Introduction

La conception devait répondre à deux questions concrètes : comment garder le lien entre une réponse et son document, et comment empêcher un utilisateur d'accéder à une fonction qui ne correspond pas à son rôle ? Les diagrammes suivants présentent les choix qui ont guidé mon développement.

3.2 Architecture logique

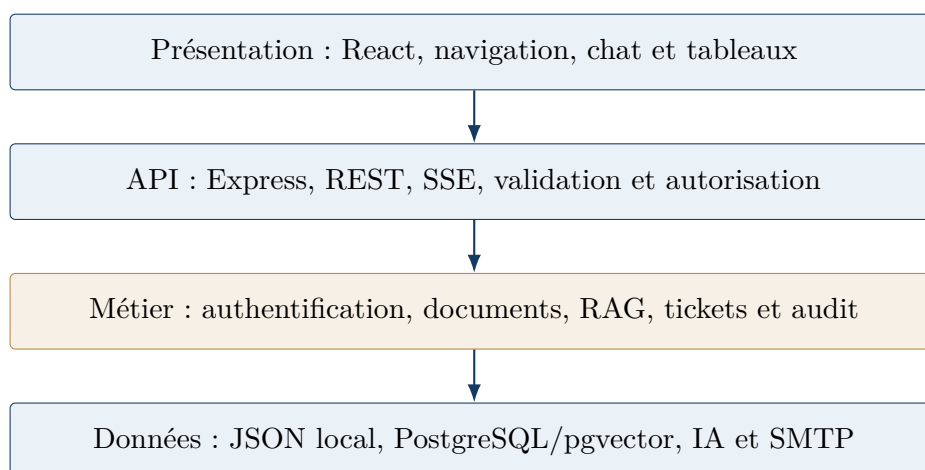


Figure 3.1 – Architecture logique en couches

La présentation masque les commandes inutiles, mais ne décide pas seule des droits. L'API vérifie chaque demande. Les services métier portent les règles et les connecteurs isolent les dépendances. Cette séparation permet de remplacer JSON par PostgreSQL sans réécrire l'interface.

3.3 Diagramme de composants

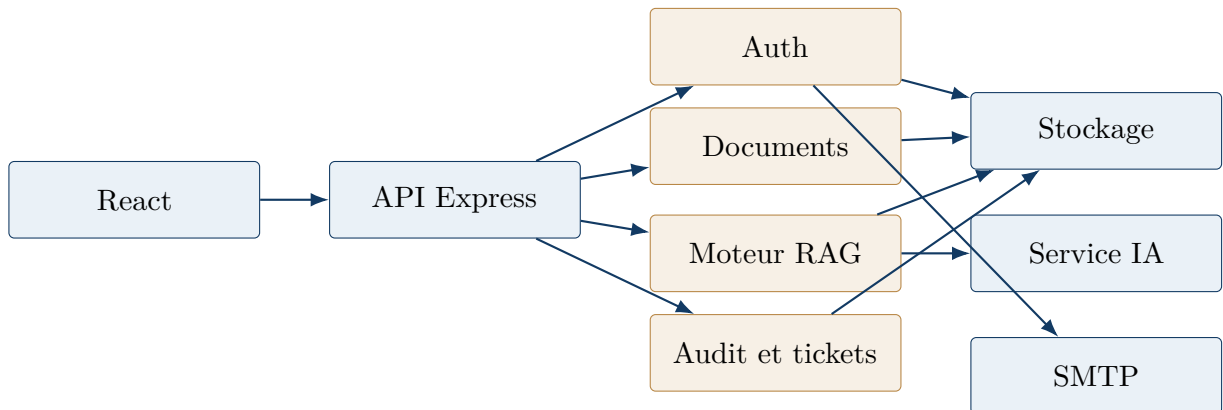


Figure 3.2 – Diagramme de composants

Le frontend ne contacte pas directement la base ni les fournisseurs privés. L'API orchestre les services et centralise la sécurité.

3.4 Modèle de domaine

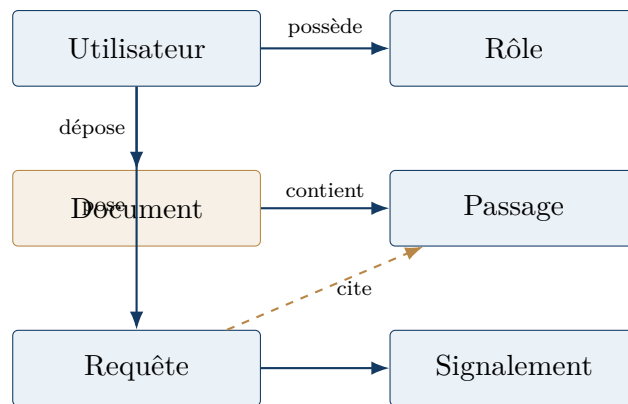


Figure 3.3 – Diagramme de classes métier simplifié

Un document conserve son auteur, son dossier parent et son état. Il produit plusieurs passages. Une requête contient question, réponse, latence et sources. Les rôles sont séparés du profil pour éviter qu'une simple modification du profil ne change les privilèges.

3.5 Modèle de données

Table 3.1 – Entités persistées

Entité	Champs	Relation
Utilisateur	email, nom, hash, TOTP	possède des rôles
Rôle	type, actif, auteur	référence un utilisateur
Document	id, nom, parent, statut, auteur	contient des passages
Passage	texte, vecteur, page	appartient à un document
Requête	question, réponse, latence	cite des passages
Signalement	numéro, type, statut	contient des commentaires
Audit	type, sévérité, détails	référence l'auteur

En mode local, les entités sont des fichiers JSON. La cible PostgreSQL reprend les mêmes responsabilités avec une colonne vectorielle.

3.6 Séquence d'authentification

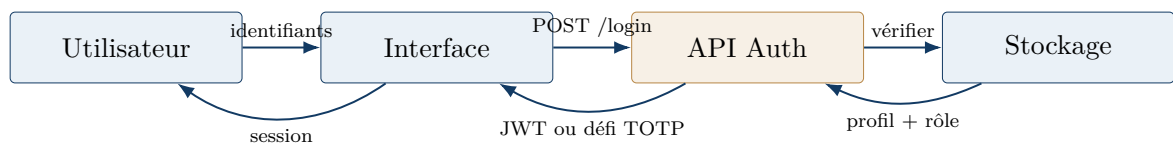


Figure 3.4 – Séquence d'authentification

Le mot de passe n'est pas conservé en clair. Si la double authentification est active, un code temporaire est demandé. À chaque requête privée, le serveur vérifie le jeton et relit le rôle actif.

3.7 Ingestion et indexation

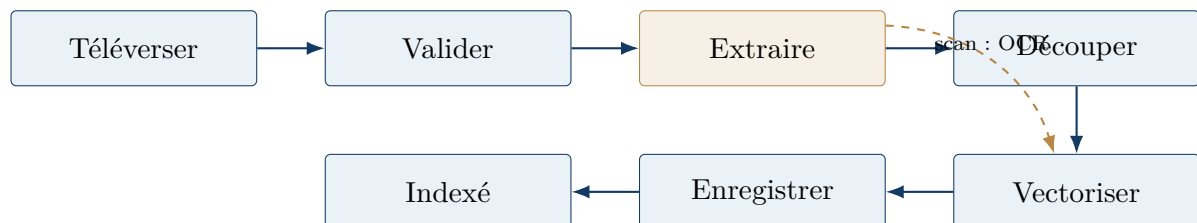


Figure 3.5 – Pipeline d'ingestion

Le parseur choisit une méthode selon l'extension. Pour un PDF, l'extraction se fait page par page. Une page presque vide déclenche l'OCR. Les passages sont assez courts pour la précision, tout en gardant un contexte utile.

3.8 Recherche hybride et RRF

La recherche combine BM25, qui favorise les mots importants, et la proximité vectorielle, qui rapproche des formulations de même sens. Le Reciprocal Rank Fusion réunit les listes sans supposer qu'elles utilisent la même échelle :

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rang}_r(d)}.$$

La constante k réduit l'effet d'une petite différence de rang. Lorsque le service est disponible, un reranker multilingue réordonne les meilleurs passages. J'ai retenu cette combinaison pour éviter de dépendre d'une seule méthode : BM25 reste efficace sur un terme précis, tandis que la recherche vectorielle aide lorsque la question reformule le texte. Elle garde aussi un fonctionnement local lorsque les services distants sont indisponibles.

3.9 Séquence d'une question RAG

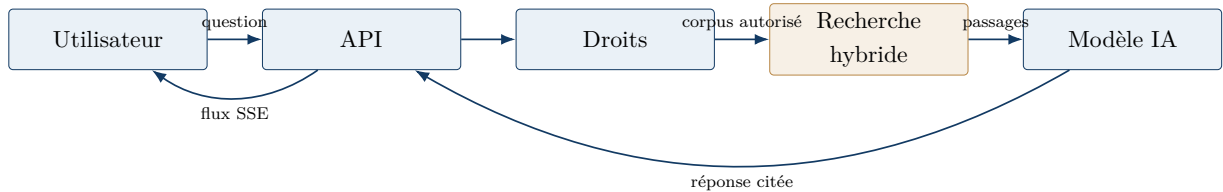


Figure 3.6 – Séquence d'une requête RAG

Si aucun passage ne dépasse le seuil, le modèle n'est pas invité à improviser. L'application indique que l'information ne figure pas dans les documents.

3.10 Contrôle d'accès par rôles

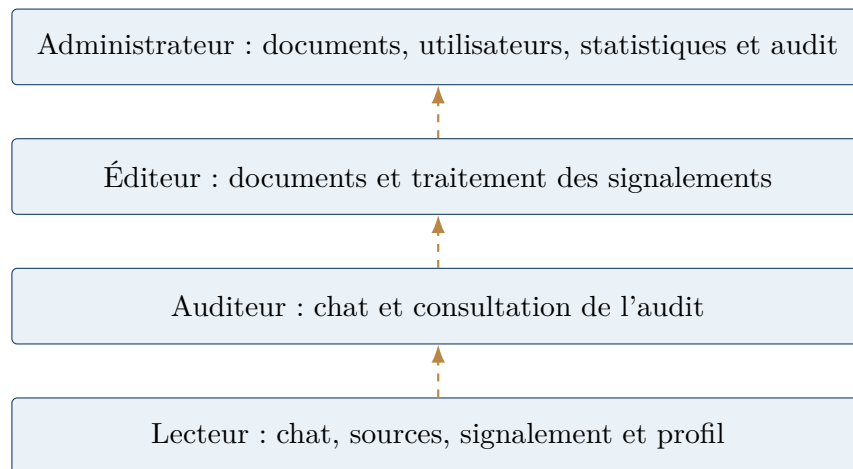


Figure 3.7 – Capacités par rôle

Chaque route déclare explicitement les rôles autorisés. L'interface masque les options inutiles, mais cette ergonomie n'est jamais utilisée comme preuve d'autorisation.

3.11 Cycle d'un signalement

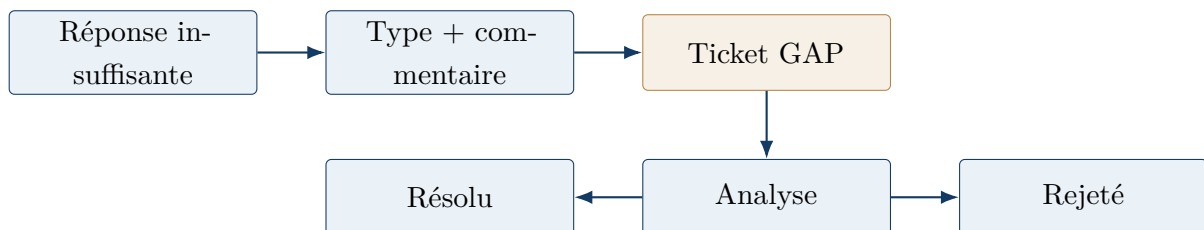


Figure 3.8 – Cycle d'une lacune documentaire

Le signalement transforme un échec en tâche d'amélioration : ajouter une procédure, corriger un fichier ou modifier l'indexation.

3.12 Architecture locale et cible

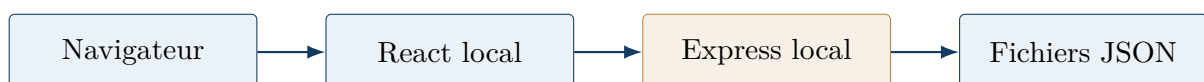


Figure 3.9 – Architecture réellement utilisée en démonstration

La cible cloud prévoit un frontend hébergé, une API, PostgreSQL/pgvector, un service IA et SMTP. Le dépôt contient aussi une cible AWS avec S3, Lambda, Bedrock, OpenSearch

et Cognito. Ces cibles sont des perspectives, pas des résultats déployés. Cette distinction clôt la conception : elle sépare clairement la réalisation locale de la cible cloud et relie chaque flux aux droits nécessaires.

Réalisation et validation

4.1 Introduction

Ce chapitre présente ce que j’ai réellement mis en oeuvre pendant le stage. Je décris d’abord l’environnement technique, puis les principaux parcours de l’application. Je termine par les difficultés rencontrées, les tests effectués et les limites de la version obtenue.

4.2 Environnement technique

Table 4.1 – Technologies utilisées

Couche	Technologie	Usage
Interface	React, Vite, TypeScript, Tailwind	Pages, formulaires et composants
API	Node.js, Express, TypeScript	Routes, sécurité et flux SSE
Documents	unpdf, Mammoth, xlsx, Tesseract.js	Extraction et OCR
Recherche	BM25, vecteurs, RRF, reranking	Classement des passages
Données	JSON local, PostgreSQL/pgvector optionnel	Persistance
Communication	REST, SSE, SMTP	Requêtes, réponse progressive et emails

Le projet est séparé en une interface et une API. Cette organisation m’a permis de tester directement les routes sensibles, sans dépendre uniquement de ce que montre l’écran. Les clés privées restent côté serveur. Le mode local utilise des fichiers JSON et un moteur

simulé afin de garder une démonstration reproductible sans compte cloud.

4.3 Authentification et gestion des profils

Au début de mon analyse, l'inscription attribuait un rôle éditeur. Ce comportement donnait trop de droits à un nouveau compte. Je l'ai remplacé par le rôle lecteur, puis j'ai vérifié que les opérations d'administration étaient refusées par l'API. La connexion contrôle le mot de passe haché et l'état du compte avant de créer la session.

L'utilisateur peut aussi activer une double authentification TOTP. Dans ce cas, le mot de passe seul ne suffit plus : un code temporaire est demandé. Pour la réinitialisation du mot de passe, un jeton limité à une heure est généré. En développement, le lien est visible pour faciliter le test ; en production, il doit être envoyé par SMTP. Cette différence est volontairement indiquée pour ne pas confondre une facilité de test et un comportement final.

4.4 Gestion et traitement des documents

L'espace documentaire permet de créer des dossiers, de déposer plusieurs fichiers, de les renommer, de les supprimer et de relancer leur indexation. J'ai conservé des interactions proches d'un gestionnaire de fichiers afin que l'utilisateur comprenne rapidement le fonctionnement. L'API limite les extensions et refuse un contenu supérieur à 20 Mo.

Chaque document passe par plusieurs états. Il est d'abord enregistré, puis marqué en cours d'indexation. Une fois le texte extrait et découpé, il devient disponible pour la recherche. Si une étape échoue, l'état d'erreur reste visible. Cette information a été importante pendant mes essais, car un fichier présent à l'écran n'est pas forcément prêt à être interrogé.

4.4.1 Extraction multi-format et OCR

Les fichiers texte, Markdown et CSV sont lus directement. Mammoth traite les documents Word et la bibliothèque xlsx parcourt les feuilles Excel. Pour un PDF, le texte est extrait page par page. Une page contenant moins de cinquante caractères est considérée comme potentiellement scannée et déclenche l'OCR lorsque celui-ci est activé.

Listing 4.1 – Décision simplifiée pour l'OCR

```
extraire le texte de la page
si longueur < 50 et OCR actif :
    lancer la reconnaissance en fran aise et en anglais
si un texte utilisable est obtenu :
    ajouter la page au document
sinon :
    conserver un avertissement
```

Ce mécanisme a amélioré la prise en charge des scans, mais il reste plus lent que l'extraction d'un PDF numérique et dépend de la qualité de l'image.

4.5 Recherche, génération et citations

Lorsqu'une question arrive, le serveur détermine d'abord le corpus autorisé. Le moteur exécute ensuite une recherche lexicale et une recherche vectorielle, puis fusionne les classements avec RRF. Si le service externe est disponible, un reranker multilingue réordonne les meilleurs passages.

Le générateur reçoit uniquement les extraits retenus et une consigne qui lui demande de s'appuyer sur ce contexte. La réponse est envoyée progressivement par SSE. Des références numérotées permettent d'ouvrir le document, la page ou l'extrait correspondant. Quand aucun passage ne dépasse le seuil choisi, l'application répond que l'information n'a pas été trouvée au lieu de compléter la réponse par une supposition.

Cette partie m'a montré qu'un assistant documentaire ne dépend pas seulement du modèle de langage. Une mauvaise extraction ou un mauvais classement produit un contexte insuffisant, même avec un bon modèle. J'ai donc accordé autant d'attention à l'ingestion et aux citations qu'à la génération.

4.6 Signalements, audit et administration

Une réponse insuffisante peut être transformée en signalement. L'utilisateur précise la catégorie, la priorité et un commentaire. Un éditeur ou un administrateur peut ensuite affecter le ticket, répondre, le résoudre ou le rejeter. Cette fonction permet de transformer un échec du moteur en action concrète sur le corpus.

Le journal d'audit conserve les événements importants et peut être filtré ou exporté. Le tableau de bord rassemble le nombre de documents, les requêtes et la latence. Les valeurs simulées restent identifiées comme telles. Enfin, l'administration des comptes est réservée au rôle administrateur et les refus sont appliqués par le serveur, même si l'interface masque déjà les commandes concernées.

4.7 Difficultés rencontrées et solutions

Table 4.2 – Difficultés rencontrées pendant le stage

Difficulté	Constat	Solution retenue
PDF hétérogènes	Un scan ne fournit presque aucun texte	Détection par seuil et recours à Tesseract.js
Services externes	Une clé ou un crédit peut manquer	Mode local et réponse fondée sur les extraits
Droits incohérents	Une option masquée ne protège pas l'API	Contrôle du rôle sur chaque route sensible
Réindexation	Le bouton ne relançait pas tout le traitement	Nouvelle extraction et vectorisation réelle
Streaming	Certains intermédiaires mettent la réponse en tampon	En-têtes SSE et événements progressifs

Ces corrections ont occupé une place importante dans mon travail. Elles m'ont appris à ne pas considérer une fonction comme terminée uniquement parce que son interface semble fonctionner.

4.8 Stratégie de validation

J'ai vérifié le parcours principal depuis l'interface, puis les règles de sécurité par des appels directs à l'API. Cette seconde vérification était nécessaire : un utilisateur peut appeler une route sans passer par les boutons prévus. Chaque scénario associe une action, un résultat attendu et le comportement observé après correction.

4.8.1 Résultats fonctionnels

Table 4.3 – Tests fonctionnels

Scénario	Résultat attendu	État
Inscription	Création avec le rôle lecteur	Validé
Connexion	Session associée au rôle actif	Validé
Réinitialisation	Jeton temporaire et préparation de l'email	Validé
Dépôt multi-format	Extraction puis indexation	Validé
PDF scanné	Déclenchement de l'OCR	Validé
Question couverte	Réponse progressive avec sources	Validé
Question hors sujet	Message prudent	Validé
Signalement	Création et suivi d'un ticket	Validé

4.8.2 Résultats de sécurité

Table 4.4 – Tests de sécurité

Scénario	Résultat attendu	État
Lecteur vers une route admin	Refus 403	Validé
Page privée sans session	Redirection vers la connexion	Validé
Modification de son propre rôle	Refus 403	Validé
Document d'un autre utilisateur	Refus 403	Validé
Téléversement sans jeton	Refus 401	Validé
Fichier exécutable ou trop volumineux	Rejet de la demande	Validé
Secret absent en production	Arrêt au démarrage	Validé

Les tests confirment le fonctionnement des parcours décrits dans ce rapport. Ils ne constituent cependant ni un audit de sécurité indépendant ni une évaluation scientifique complète de la qualité des réponses.

4.9 Mode d'exécution et déploiement envisagé

La version validée pendant le stage fonctionne localement avec une interface sur le port 5173 et une API sur le port 3001. Ce choix est pratique pour la démonstration et permet de travailler sans dépendre en permanence d'un service distant.

Une mise en ligne est documentée avec un hébergement séparé pour l'interface et l'API, une base PostgreSQL avec pgvector, un service d'IA et SMTP. Le dépôt contient aussi

une cible AWS fondée sur S3, Lambda, Bedrock et OpenSearch. Je les présente comme des architectures envisagées : aucun déploiement public complet n'a été validé pendant le stage.

Avant une utilisation réelle, il faudrait stocker les secrets chez l'hébergeur, limiter précisément les origines autorisées, activer les sauvegardes et vérifier les réponses 401 et 403. Il faudrait également tester l'envoi réel des emails et surveiller les erreurs.

4.10 Limites et perspectives

Le stockage JSON est adapté à une démonstration mais pas à plusieurs utilisateurs travaillant simultanément. L'OCR peut ralentir le traitement de nombreux scans. Le reranking dépend d'un service externe et les tests d'interface restent moins développés que les tests d'API. Enfin, l'évaluation de la recherche ne repose pas encore sur un jeu de questions de référence.

Les premières évolutions seraient donc de passer à PostgreSQL et pgvector, d'ajouter des tests de bout en bout pour chaque rôle et de constituer un corpus d'évaluation. Pour chaque question de référence, il serait alors possible de mesurer si le bon passage est retrouvé, si la réponse respecte la source et si la citation est utile. Une file de traitement pourrait aussi répartir les tâches OCR plus longues.

4.11 Conclusion

La réalisation obtenue couvre le parcours essentiel d'une application RAG et plusieurs fonctions de sécurité. Les tests ont surtout servi à trouver puis corriger des écarts concrets. La version locale reste un prototype de stage : elle est démontrable et structurée, mais son industrialisation constitue une étape future.

Conclusion générale

Au terme de ces cinq semaines, j'ai obtenu une version démontrable d'une application d'intelligence documentaire. Le besoin de départ était courant : retrouver une information précise dans des documents dispersés. La réponse proposée combine extraction, indexation, recherche hybride, génération contrôlée et citations consultables.

La conception a été essentielle. Les diagrammes ont séparé l'interface, l'API, le moteur RAG et les données. Ils ont rendu visibles les flux sensibles : authentification, ingestion, question, rôles et signalements. Cette vue d'ensemble a facilité les corrections découvertes pendant la validation.

J'ai travaillé avec React, Express et TypeScript, mais aussi avec SSE, embeddings, RRF, OCR et TOTP. J'ai compris qu'une bonne réponse dépend d'abord de la qualité du corpus et de la recherche. J'ai également appris qu'une interface claire ne garantit pas la sécurité : l'autorisation doit être contrôlée côté serveur.

La version obtenue fonctionne localement et couvre le parcours principal : ajout d'un document, indexation, question, réponse sourcée et contrôle des droits. J'ai aussi relié l'OCR, les signalements, l'audit et la réinitialisation du mot de passe. L'infrastructure AWS et le déploiement public restent des perspectives ; je ne les présente donc pas comme des résultats du stage.

Cette expérience a renforcé ma capacité à analyser un projet existant, organiser les priorités, documenter les choix et vérifier un résultat. Elle m'a surtout montré qu'un projet d'intelligence artificielle professionnel doit rester transparent sur ses sources, ses limites et son état réel de déploiement.

Bibliographie

- [1] P. Lewis et al., « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks », NeurIPS, 2020.
- [2] OWASP Foundation, *Authentication Cheat Sheet*, https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, consulté en septembre 2026.
- [3] Meta Open Source, *React Documentation*, <https://react.dev/>.
- [4] OpenJS Foundation, *Express Documentation*, <https://expressjs.com/>.
- [5] A. Kane, *pgvector*, <https://github.com/pgvector/pgvector>.
- [6] Tesseract contributors, *Tesseract OCR Documentation*, <https://tesseract-ocr.github.io/>.

Scénario de démonstration

La démonstration dure quinze à vingt minutes. Elle commence par la page publique et la connexion avec un compte administrateur. Dans Documents, on crée un dossier « Ressources Humaines », puis on ajoute un fichier. Dans le chat, une question couverte montre le flux et les citations. Une question hors sujet montre la prudence. Le tableau de bord et l'audit terminent le parcours.

Le message principal est que l'application cherche avant de répondre et conserve la source utilisée. Les services AWS affichés en mode mock sont présentés comme une cible, non comme un déploiement actif.

Matrice détaillée des droits

Fonction	Lecteur	Éditeur	Auditeur	Admin
Chat et citations	Oui	Oui	Oui	Oui
Créer un signalement	Oui	Oui	Oui	Oui
Téléverser	Non	Oui	Non	Oui
Traiter un ticket	Non	Oui	Non	Oui
Audit global	Non	Non	Oui	Oui
Gérer les rôles	Non	Non	Non	Oui
Statistiques	Non	Non	Non	Oui

Informations à compléter

1. Insérer les logos officiels de l'ENI Carthage et SMARTOVATE.
2. Ajouter le nom et la fonction de l'encadrant académique.
3. Ajouter le nom et la fonction de l'encadrant professionnel.
4. Personnaliser les remerciements.
5. Confirmer l'année universitaire demandée.
6. Ajouter éventuellement des captures réelles en annexe.